

Midsem Solutions

March 10, 2026

Question 1 (10 Marks). Suppose we are given an array $B[1 \dots n]$ of n boolean values, which is supposed to be sorted (all Falses before all Trues) but isn't. An inversion in B is a pair of indices (i, j) such that $i < j$ and $B[i] = \text{True}$ and $B[j] = \text{False}$.

- (a) [4 Marks] Describe and analyze an algorithm to compute the number of inversions in the input array B in $O(n)$ time. [Marks distribution: 2 Marks for the algorithm, and 2 marks for running time and correctness analysis.]
- (b) [6 Marks] The length of an inversion (i, j) is the difference $j - i$. By definition, the length of an inversion is an integer between 1 and $n - 1$. Describe and analyze an algorithm to compute, for every integer $1 \leq \ell \leq n - 1$, the number of inversions of length ℓ in the input array B in $O(n \log n)$, and analyze the correctness. [Marks distribution: 2 Marks for the algorithm, and 4 marks for running time and correctness analysis.]

For example, if the input array is $B = [\text{T}, \text{F}, \text{F}, \text{T}, \text{F}, \text{F}, \text{T}]$, your first algorithm should return the integer 6, and your second algorithm should return the array $[2, 2, 0, 1, 1, 0]$.

Solution. (a) **Algorithm:** Iterate through B from index 1 to n . Maintain a running count of True values encountered, denoted as t . Initialize $total_inversions = 0$. For each element, if $B[k] = \text{True}$, increment t . If $B[k] = \text{False}$, add t to $total_inversions$. Return $total_inversions$.

Analysis: For every False encountered at index j , the variable t exactly tracks the number of preceding True values where $i < j$. By adding t to the total at each False, the algorithm correctly counts every valid inversion ending at j exactly once. The algorithm performs a single linear scan of the array with $O(1)$ operations per element.

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

(b) **Algorithm:** We model the distance between True and False values as a polynomial convolution.

- (a) Construct an array A of size n such that $A[i] = 1$ if $B[i] = \text{True}$, and 0 otherwise. Define the polynomial $P(x) = \sum_{i=0}^{n-1} A[i]x^i$.
- (b) Construct an array C of size n such that $C[k] = 1$ if $B[n - 1 - k] = \text{False}$, and 0 otherwise. Define the polynomial $Q(x) = \sum_{k=0}^{n-1} C[k]x^k$.
- (c) Compute the polynomial product $R(x) = P(x)Q(x) = \sum_{m=0}^{2n-2} c_m x^m$ using the Fast Fourier Transform (FFT).
- (d) For each $1 \leq \ell \leq n - 1$, the number of inversions of length ℓ is given by the coefficient $c_{n-1-\ell}$ of $R(x)$. Return these coefficients.

Analysis: The coefficient c_m computes $\sum A[i]C[k]$ where $i + k = m$. Substituting $k = n - 1 - j$, we get $i + n - 1 - j = m$, which simplifies to $j - i = n - 1 - m$. Let the inversion length be $\ell = j - i$. This yields $\ell = n - 1 - m$. Thus, $c_{n-1-\ell}$ strictly counts valid pairs (i, j) at distance ℓ where $B[i] = \text{True}$ and $B[j] = \text{False}$.

- **Time Complexity:** $\mathcal{O}(n \log n)$ to compute the polynomial product via FFT. Array construction and coefficient extraction take $\mathcal{O}(n)$ time.
- **Space Complexity:** $\mathcal{O}(n)$ to store the polynomials and evaluate the FFT.

□

Rubric: Part(a) 1 marks if suboptimal algorithm. 2 Marks for the algorithm, and 1 marks for running time and 1 for correctness analysis. Part(b) 2 Marks for the algorithm, and 4 marks for running time and correctness analysis (1 + 3). 1 mark if $\mathcal{O}(n^2 \log n)$, 2 marks if $\mathcal{O}(n^2)$ algo provided.

Question 2 (8 Marks). You have a large number of assignments in this semester, each of which takes one full day to finish (if you are not doing any other assignment on that day). The i -th assignment is released on a particular day, say r_i and has a deadline d_i . That means you can schedule the i -th assignment on any day t such that $r_i \leq t \leq d_i$. Given $r_1, r_2, \dots, r_n$ and $d_1, d_2, \dots, d_n$, you want to find out whether it is possible to make a schedule to finish all the assignments on time. Give a polynomial-time algorithm for this, analyze the running time, and prove the correctness of your algorithm. [Marks distribution: 3 marks for the algorithm, 2 marks for the running time analysis, and 3 marks for the correctness proof of the algorithm.]

Hint. A greedy approach may work. A typical approach to prove correctness of a greedy algorithm is as follows. You want to prove that if there is a feasible schedule then the greedy algorithm will give a feasible schedule. Consider a feasible schedule that agrees with first $i - 1$ choices of your greedy schedule. Modify the feasible schedule to make it agree on first i choices, while maintaining feasibility. Then argue inductively.

Solution. Algorithm.

Repeatedly (for each day t) pick an available (released, unscheduled) job with smallest deadline and schedule it on t . If none are available, jump t to the next release day. (Use a min-heap keyed by d_i after sorting jobs by r_i .)

Running time.

Sorting $\mathcal{O}(n \log n)$; each job is pushed/popped once from the heap so heap work $\mathcal{O}(n \log n)$. Total $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ space.

Correctness.

Let G be the greedy schedule and assume some feasible schedule S exists. By induction on scheduled positions: suppose S agrees with G on the first $k - 1$ days. Let t be the k -th day in G and let j be the job G places at t (it has minimum deadline among available jobs at t). If S already has j at t we are done; otherwise S places some k_0 at t and places j later at $s > t$. Swap the jobs at t and s . Releases hold since $r_j \leq t \leq s$ and $r_{k_0} \leq t \leq s$. Deadlines hold because originally $s \leq d_j$ and by greedy minimality $d_j \leq d_{k_0}$, so $s \leq d_{k_0}$. Hence the swap preserves feasibility and makes S agree with G on one more day. Repeating yields a feasible schedule that equals G . If greedy ever finds a job with $d_j < t$ it reports infeasible, which is correct. □

Rubric: 3 marks for providing a polynomial-time algorithm for the problem, 2 marks for giving the time complexity analysis of the algorithm, and 3 marks for the proof of correctness

of the algorithm.

Question 3 (10 Marks). Prof. Strange is cooking using ingredients from a garden arranged in a grid with n rows and m columns. Each grid cell (i, j) (for $1 \leq i \leq n$ and $1 \leq j \leq m$) contains an ingredient with a *tastiness* value given by a positive number $T_{i,j}$.

Prof. Strange does not like cooking “by the book.” Hence, to prepare dinner, Prof. stands at a grid cell (i, j) and selects one ingredient from each of the four quadrants defined relative to that cell. The tastiness of the dish is the product of the tastiness values of the four selected ingredients.

Design an $O(nm)$ dynamic programming algorithm to maximize the tastiness of the dish. Here the four **quadrants** relative to a cell (i, j) are defined as follows:

$$\begin{aligned}\text{top-left} &= \{(a, b) \mid a < i, b < j\}, \\ \text{bottom-left} &= \{(a, b) \mid a > i, b < j\}, \\ \text{top-right} &= \{(a, b) \mid a < i, b > j\}, \\ \text{bottom-right} &= \{(a, b) \mid a > i, b > j\}.\end{aligned}$$

Since all four quadrants must be non-empty, the choice of cell (i, j) is restricted to indices satisfying $1 < i < n$ and $1 < j < m$.

- (a) [6 Marks] Define $TL_{i,j}$ to be the maximum tastiness value in the top-left quadrant relative to cell (i, j) : $TL_{i,j} = \max\{T_{a,b} \mid 1 \leq a < i, 1 \leq b < j\}$. Design a dynamic programming algorithm to compute $TL_{i,j}$ for all $1 < i < n$ and $1 < j < m$ in $O(nm)$ time. [Marks distribution: 2 marks for the algorithm, 2 marks for the running time analysis, and 2 marks for the correctness proof of the algorithm.]
- (b) [4 Marks] Using the idea from part (a), design an $O(nm)$ algorithm to compute the maximum possible tastiness of the dish. [Marks distribution: 2 marks for the algorithm, 2 marks for the running time analysis and correctness proof of the algorithm.]

Solution. (a) When trying to calculate $TL_{i,j}$, we see that the maximum can be at cell $(i-1, j-1)$. If not, it must lie either in the rectangle from $(1, 1)$ to $(i-1, j-2)$, or the rectangle from $(1, 1)$ to $(i-2, j-1)$, or both. These three overlapping cases cover our required rectangle. We have then,

$$TL_{i,j} = \max\{T_{i-1,j-1}, TL_{i-1,j}, TL_{i,j-1}\}$$

For the base cases, we can just set $TL_{0,j} = TL_{i,0} = 0$ for all valid values of i and j . We can compute the DP value for each state in $O(1)$ time. There are nm states, so our algorithm is $O(nm)$.

- (b) In part (a), we calculated the range maximum for the top-left quadrant. We can similarly define range maximums for the other quadrants. Let $BL_{i,j} = \max\{T_{a,b} \mid i < a \leq n, 1 < b \leq j\}$, $TR_{i,j} = \max\{T_{a,b} \mid 1 \leq a < i, j < b \leq m\}$, and $BR_{i,j} = \max\{T_{a,b} \mid i < a \leq n, j < b \leq m\}$. Each of these can be computed in $O(nm)$ time, similar to TL .

To calculate the tastiest dish Prof. Strange can cook when she stands at cell (i, j) ($1 < i < n$ and $1 < j < m$), we now just need to compute the product $TL_{i,j}BL_{i,j}TR_{i,j}BR_{i,j}$ and pick the maximum product. This can be done in $O(nm)$ time.

□

Rubric: Complete marks have been given for any correct solution with $O(mn)$ complexity with proper justification. Partial marks have been given for minor indexing errors. For part (b), 1 mark has been cut for accidentally taking max/sum of tastiness.

Question 4 (6 Marks). Consider an instance of the *Edit Distance* problem, where strings are made of English alphabet, and the *edit* costs are as follows.

- inserting or deleting a character – cost 2
- substituting a character with a different character
 - if the two characters are adjacent on the keyboard, then – cost 1 (see Figure 1, where edges show the pairs of characters which are adjacent)
 - otherwise – cost 3

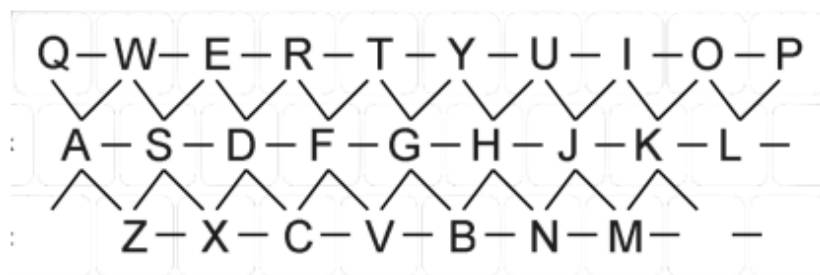


Figure 1: Keyboard with edges showing the adjacent characters.

Find a way to modify string LINUX to DOUBT with cost at most 10. You should clearly show the sequence of edit steps with their costs. [Marks policy: Only 3 marks for cost 11 and only 1 marks for any higher cost].

Solution.

$$\begin{aligned} \text{LINUX} &\xrightarrow{\text{Insert D, 2}} \text{DLINUX} \xrightarrow{L \rightarrow O, 1} \text{DOINUX} \xrightarrow{I \rightarrow U, 1} \text{DOUNUX} \\ &\xrightarrow{N \rightarrow B, 1} \text{DOUBUX} \xrightarrow{U \rightarrow T, 3} \text{DOUBTX} \xrightarrow{\text{Delete X, 2}} \text{DOUBT} \end{aligned}$$

$$\text{Total cost} = 2 + 1 + 1 + 1 + 3 + 2 = \boxed{10}.$$

□

Question 5 (10 Marks). At Wog Hearts School of witchcraft and wizardry, n incoming students are sorted into four houses by an ancient magical artifact called the Sorting Sock. The Sorting Sock first sorts the n students by each of the four houses' attributes and then uses the results to make its determinations. For each of the following parts, state and justify what type of *sort* would be most efficient. [Marks policy: Provide a clear runtime analysis for each part.]

1. [2 Marks] For House Puffle Huff, students must be sorted by **friend number**, i.e., how many of the other $n - 1$ incoming students they are friends with, which can be determined in $O(1)$ time.

2. [2 Marks] For House Craven Law, students must be sorted by the weight of their *books*. Book weights cannot be measured precisely, but the Sorting Sock has a *scale* that can determine in $O(1)$ time whether one set of books has total weight greater than, less than, or equal to another set of books.
3. [2 Marks] For House Driven Gore, students must be sorted by *bravery*, which can't be directly measured or quantified, but for any set of students, the Sorting Sock can determine the bravest among them in $O(1)$ time, e.g., by presenting the students with a scary situation.
4. [4 Marks] For House Leather Skin, students must be sorted by their *magical lineage*: how many of a student's ancestors within the previous $3\lceil\log n\rceil + 4$ generations were magical. Recall that humans, magical or not, always have two parents in the previous generation, unlike binary tree nodes which have at most one. Assume the Sorting Sock can compute the magical lineage of a student in $O(1)$ time.

Assessment Rubric: For Parts 1, 2, and 3, 1 mark will be awarded for the implementation of any less efficient sorting technique. A deduction of 0.5 marks will be applied if a proper justification for the chosen technique's efficiency is missing.

For Part 4, 2 marks will be awarded for the accurate calculation of wizard ancestors, and 1 mark will be awarded for the implementation of any less efficient sorting technique.

Solution. 1. Friend numbers are non-negative integers less than n , so we can use counting sort to sort the students in worst-case $O(n)$ time. (Radix sort also works with the same running time.) Since we have to compute friend number for each student, any algorithm will take at least $\Omega(n)$ time, so this is optimal.

Common Mistakes: Using a sort that is not $O(n)$

2. A scale weighing is a comparison with a constant number of outcomes, so the comparison sort $\Omega(n \log n)$ lower bound applies. So we cannot do better than by using an worst-case $O(n \log n)$ sorting algorithm, e.g., merge sort, using the scale to compare one student's books against another's.

Common Mistakes: Insufficient justification for why $O(n \log n)$ is optimal

3. We can't quantify bravery, so we can't hope to use any integer-based algorithms. However, the Sorting Sock can find a student of maximum bravery in $O(1)$ time, so we repeatedly find and select a bravest student among all previously unselected students in worst-case $O(n)$ time, which is again optimal.

Common Mistakes: Arguing a $\Omega(n \log n)$ lower bound

4. Each student has at most 2^k ancestors in the k th generation preceding. Thus, the number of wizard ancestors will be a non-negative number bounded above by $\sum_{k=1}^{3\lceil\log n\rceil+4} 2^k < 2^{3(\log n+1)+5} = 2^8 2^{3\log n} = 2^8 n^{3\log 2} = O(n^c)$ for any $c > 3\log 2$. Thus, we can use radix sort to sort the students by their magical lineage in worst-case $O(n)$ time, which is again optimal.

Common Mistakes:

- Claiming $3\lceil\log n\rceil + 4$ ancestors instead of $O(2^{3\lceil\log n\rceil+4})$
- Saying that $2^{3\lceil\log n\rceil+4} = O(n)$ and using counting sort
- Using a sort that is not $O(n)$.

□

Question 6. Recall the definition of Fourier transform. Let ω be a primitive d -th root of unity. For a given length- d tuple $(a_0, a_1, \dots, a_{d-1})$, we want to compute d values, i.e.,

$$a_0 + a_1\omega^i + a_2\omega^{2i} + \dots + a_{d-1}\omega^{(d-1)i},$$

for each $0 \leq i \leq d-1$.

We had discussed a divide-and-conquer approach for fast Fourier transform. It is possible to do an iterative implementation of that approach, which uses only $O(d)$ size memory. For simplicity assume $d = 2^r$. Consider the following pseudocode:

1. Let A be a length d array initialized as $[a_0, a_1, \dots, a_{d-1}]$.
2. **for** $k = 1$ **to** r **{**
3. **for** $i = 0$ **to** $(2^{r-k} - 1)$ **{**
4. **for** $j = 0$ **to** $(2^{k-1} - 1)$ **{**
5. define $\text{index1} = i \times 2^k + j$ and $\text{index2} = \text{index1} + 2^{k-1}$.
6. Initialize two temporary variables temp1 and temp2 .
7. Compute temp1 and temp2 using $A[\text{index1}]$ and $A[\text{index2}]$.
8. Copy temp1 into $A[\text{index1}]$ and temp2 into $A[\text{index2}]$.
9. **}**
10. **}**
11. **}**

[3+3 marks]. Describe how will you compute temp1 and temp2 in line 7.

Marks awarded to everyone due to an error in the question.

Bonus Question [5 Marks]. Given a convex polygon $\mathcal{P}$ as an array of its n vertices in sorted order along the boundary. Show that, given a query point q , it can be tested in time $O(\log n)$ whether q lies inside $\mathcal{P}$.

Solution. Assume $\mathcal{P}$ is a convex polygon whose vertices are given in counterclockwise order without loss of generality. Choose the vertex with the smallest x -coordinate; if there are multiple such vertices, select the one with the smallest y -coordinate among them. Denote this vertex by p_0 .

Because the polygon is convex and listed counterclockwise, the remaining vertices $p_1, p_2, \dots, p_{n-1}$ are sorted by increasing angle with respect to p_0 .

Observe that if a query point p lies inside $\mathcal{P}$, then it must lie in at least one of the triangles

$$\triangle(p_0, p_i, p_{i+1}), \quad 1 \leq i \leq n-2.$$

If p lies on a boundary edge, it may belong to two adjacent triangles. Among all such triangles containing p , consider the one with the largest index i .

Algorithm.

First check whether p lies between the rays p_0p_1 and p_0p_{n-1} . If not, then p is outside $\mathcal{P}$.

This can be verified by checking that the conditions

$$(p_1 - p_0) \times (p - p_0) \geq 0 \quad \text{and} \quad (p_{n-1} - p_0) \times (p - p_0) \leq 0$$

hold. If either condition fails, p cannot lie inside the polygon.

Assuming p lies within the outer wedge, we perform a binary search over the vertices $p_1, \dots, p_{n-1}$ to find the largest index i such that

$$(p_i - p_0) \times (p - p_0) \geq 0.$$

Since the vertices are sorted by polar angle around p_0 , the sign of this expression changes monotonically, which makes binary search applicable.

Once the index i is found, we test whether p lies inside or on the boundary of the triangle $\triangle(p_0, p_i, p_{i+1})$.

Time Complexity.

The first check takes constant time. The binary search over the vertices takes $O(\log n)$ time. The final triangle test takes constant time. Therefore, the overall running time is

$$O(\log n).$$

Rubric: 3 marks for the Binary Search component, in which 2 marks for the correctness of the Binary Search algorithm idea and 1 mark for its formulation in terms of cross product or any mathematical equivalent. 2 marks for the triangle check at the end, in which 1 mark for mentioning that the triangle interior needs to be explicitly checked and 1 mark for its formula. If the binary search doesn't work, no marks for the triangle check. In the Binary search component, sometimes the algorithm mentioned doesn't consider all edge cases. In those cases, we have been strict and haven't awarded the full 2 marks.

□